

Game tech

bachelor in de elektronica – ICT – Brugge

docent: Van Gaever T.

academiejaar 2026-2027

Opdracht 4: LILYGO T8 ESP32-S2

Opdracht: LILYGO T8 ESP32-S2

In opdracht 3 heb je een transparante BLE-UART bridge gebouwd met een ESP32-C3 LilyGO-module op USART1. Via een telefoon-app kon je commando's versturen om RC5-instellingen te configureren, hit-data op te vragen en de hit-teller te resetten. De debug-uitvoer naar PuTTY liep los daarvan over USART2 via de ST-Link VCP.

In deze opdracht breiden we dat systeem uit met een visuele speler-HUD. Een tweede LilyGO-bord — het LilyGO T8 met een ingebouwd TFT-scherm — wordt aan de opstelling toegevoegd. Op dat scherm verschijnt de spelernaam, de teamkleur als achtergrond en een grafische weergave van het aantal resterende hitpoints. Bij elke hit verdwijnt er zichtbaar één hitpoint.

Het TFT-scherm is géén terminal: de gebruiker ziet daar enkel wat hij als speler op het veld moet weten. Het verkeer tussen de BLE-app en de STM32 (commando's en hun bevestigingen) verschijnt dus niet op het scherm — die informatie blijft zichtbaar in de telefoon-app zelf. Wat wél naar het scherm wordt gestuurd, zijn uitsluitend updates van naam, teamkleur en hitpoint-stand.

1. Inleiding

- De BLE-module op USART1 blijft volledig ongewijzigd functioneel: hits, reset en bestaande commando's blijven langs daar lopen.
- Er wordt een tweede LilyGO (met TFT) toegevoegd op USART2, met DMA voor de verzending. Die lijn wordt gedeeld met de ST-Link VCP debug-uitgang — de LilyGO luistert passief mee.
- De commandoparser uit opdracht 3 wordt uitgebreid met drie nieuwe commando's (zie verder).
- De STM32 houdt een interne hitpoint-teller bij die daalt bij een geldige hit. Bij elke wijziging wordt het TFT via USART2 op de hoogte gebracht.

De rest — het RC5-protocol, de hit-teller uit opdracht 3, de bestaande parser-logica — blijft onveranderd.

2. Hardwareopstelling

De opstelling bestaat uit de STM32 als centrale controller, met daaromheen drie bestaande blokken plus één nieuwe:

- De ESP32-C3 BLE-module blijft aangesloten op USART1 zoals in opdracht 3 (ongewijzigd).
- De debug-uitgang van de STM32 loopt via USART2 naar de ST-Link VCP, zoals in opdracht 2 en 3 (ongewijzigd).
- De IR-zender en -ontvanger voor RC5 blijven op hun bestaande timer-pinnen (ongewijzigd).

- Nieuw: de LilyGO T8 met TFT wordt aangesloten op de TX-lijn van USART2. Hij deelt die lijn met de ST-Link VCP en functioneert daarop enkel als ontvanger (RX). Er hoeft geen retourlijn te zijn.

Samenvatting van de UART-rollen

Onderdeel	Beschrijving
USART1	BLE-module (zoals opdracht 3) — hits, reset, commando's, bevestigingen
USART2 TX	Gedeeld: debug-uitvoer naar ST-Link VCP én HUD-updates naar het TFT
USART2 RX	Enkel van de ST-Link VCP; het TFT stuurt niets terug

Gevolg van de gedeelde lijn

Omdat het TFT en de VCP op dezelfde fysieke TX-lijn van de STM32 hangen, ontvangt het TFT álles wat de STM32 uitstuurt — inclusief gewone debug-uitvoer. Om te vermijden dat die debug-regels het scherm in de war sturen, is een markering nodig die HUD-updates duidelijk van debug-regels onderscheidt. De LilyGO-firmware negeert alle regels die níet als HUD-update gemarkeerd zijn. Zo blijft PuTTY tegelijk bruikbaar voor debug en blijft het scherm voorspelbaar.

3. Uitbreiding van het commandosysteem

De bestaande commando-parser uit opdracht 3 wordt uitgebreid met drie nieuwe commando's. De exacte naamgeving, syntax en parameter-conventies mag je zelf kiezen, mits consistent met het bestaande commando-ontwerp (tekstueel, lijn-gebaseerd, met duidelijke bevestiging).

3.1. Nieuwe commando's (van telefoon-app naar STM32)

1. Een commando dat de spelernaam instelt. Parameter: een korte tekststring (maximale lengte zelf te bepalen, richtwaarde ca. 16 karakters). Gevolg: de nieuwe naam wordt onmiddellijk op het TFT getoond.
2. Een commando dat de teamkleur instelt. Parameter: een kleurwaarde in een formaat naar keuze (bv. hex-notatie). Gevolg: de achtergrondkleur van het TFT verandert onmiddellijk naar de gekozen kleur.

3. Een commando dat een hit simuleert. Geen parameter. Gevolg: het aantal hitpoints daalt met 1 (niet onder 0) en het TFT toont één hitpoint minder. Dit is functioneel equivalent aan een echte IR-hit en laat toe het scherm te testen zonder IR-hardware. De bestaande commando's uit opdracht 3 (hit-opvragen, reset, adres instellen, ...) blijven ongewijzigd werken. Bij een reset-commando moeten de hitpoints bijkomend teruggezet worden op hun maximum en moet het TFT daar bewust van worden gemaakt.

3.2. Commando's van STM32 naar het TFT?

De STM32 stuurt updates naar het TFT uitsluitend in reactie op gebeurtenissen die de speler op het scherm moet zien. Drie soorten updates volstaan:

- Een update van de spelernaam, uitgestuurd zodra een nieuw naam-commando via BLE werd verwerkt.
- Een update van de teamkleur, uitgestuurd zodra een nieuw kleur-commando werd verwerkt.
- Een update van het resterende aantal hitpoints, uitgestuurd na een geldige IR-hit, na een simulatie-commando, en na een reset.

Elke update is een korte, afgebakende tekstregel met een herkenbaar voorvoegsel (zelf te kiezen) zodat de LilyGO-firmware ze duidelijk kan onderscheiden van gewone debug-uitvoer. Regels zonder dat voorvoegsel worden door het scherm genegeerd maar blijven wél zichtbaar in PuTTY — wat handig is tijdens het debuggen.

Belangrijk: het TFT hoeft niet te weten waaróm het aantal hitpoints daalt. Een IR-hit, een simulatie-commando en een reset leiden alle drie tot dezelfde soort hitpoint-update. De STM32 is de enige partij die de hitpoint-logica kent (ondergrens, maximum, toestand); het TFT toont enkel wat het krijgt meegedeeld.

4. Opdracht 1: USART2 met DMA opzetten voor het TFT

Configureer de STM32 zodat ze HUD-updates kan versturen via USART2 met DMA, zonder dat de CPU geblokkeerd wordt tijdens de verzending. De bestaande debug-omleiding naar USART2 (printf) moet blijven werken. Bedenk hoe je HUD-updates en debug-uitvoer op dezelfde peripheral laat samenwerken zonder dat ze elkaar in de war sturen.

Test dat bij opstart een eerste HUD-update correct op PuTTY verschijnt als leesbare tekstregel. Als de LilyGO al geflasht is (zie volgende opdracht), moet dezelfde regel ook zichtbaar effect op het scherm hebben.

5. Opdracht 2: TFT-firmware verkennen en flashen

Voor de LilyGO T8 krijg je een voorbereide schets aangeleverd. Je hoeft ze niet zelf te schrijven; wel moet je begrijpen wat ze doet en ze correct op het bord flashen.

De schets implementeert kort gezegd:

- Ontvangst van tekstregels over UART, met regelmatige parsing op het herkenbare voorvoegsel.
- Drie soorten render-routines: één voor de spelernaam, één voor de achtergrondkleur, één voor de hitpoint-weergave.
- Afhandeling van kleurconversie naar het paneelformaat van de ST7789.

Installeer de nodige toolchain, controleer de pin-configuratie van de seriële poort op de geleverde schets, flash het bord en verifieer dat het TFT een default-weergave toont bij opstart.

6. Opdracht 3: Commando's verbinden met HUD-updates

Centraliseer de spelertoestand in één enkele datastructuur in RAM: minstens de naam, de teamkleur, het huidige aantal hitpoints, het maximum aantal hitpoints, en het eigen RC5-adres uit opdracht 3. Initialiseer met zinvolle defaults bij opstart en stuur meteen de overeenkomstige HUD-updates, zodat het scherm vanaf tijdstip nul correct is.

Breid vervolgens de commando-parser uit zodat:

- een naam-instel-commando de naam in de datastructuur aanpast en de overeenkomstige HUD-update verstuurt;
- een kleur-instel-commando de teamkleur aanpast en de overeenkomstige HUD-update verstuurt;
- een hit-simulatie-commando hetzelfde pad volgt als een echte IR-hit: hitpoints verlagen (met ondergrens 0), en de HUD-update versturen;
- een echte IR-hit op het eigen RC5-adres hetzelfde effect heeft;
- een reset-commando de hitpoints weer op het maximum zet en de overeenkomstige HUD-update verstuurt.

Zorg dat antwoorden op BLE-commando's (OK, fouten, opgevraagde hit-waarden, ...) uitsluitend via USART1 teruggestuurd worden naar de app. Ze mogen niet op USART2 verschijnen — dan zou het TFT ze te zien krijgen (weliswaar negeren door het ontbreken van het voorvoegsel), maar ze zouden de debug-uitvoer in PuTTY nodeloos vervuilen.

7. Opdracht 4: End-to-end test

Test de volledige keten in de volgende volgorde, en verzamel onderweg bewijsmateriaal voor je verslag.

4. Start alle drie de boards op (STM32, BLE-module, LilyGO met TFT). Bij opstart moet het TFT de default-naam en default-kleur tonen, met alle hitpoints gevuld.
5. Connecteer met de BLE-app met de ESP32-C3 (zoals in opdracht 3).
6. Stuur een naam-commando — de nieuwe naam verschijnt op het TFT.
7. Stuur enkele kleur-commando's met verschillende waarden — de achtergrondkleur verandert telkens.
8. Stuur enkele hit-simulatie-commando's na elkaar — telkens verdwijnt er één hitpoint op het scherm.
9. Stel je eigen RC5-adres in en laat een andere IR transmitter je raken met de IR-zender — elk raak schot laat opnieuw één hitpoint verdwijnen (hetzelfde gedrag als bij de simulatie).

10. Stuur een reset-commando — alle hitpoints keren terug.
11. Controleer in PuTTY dat je zowel de gewone debug-regels ziet als de HUD-updates.
12. Controleer in de BLE-app dat je dáár géén HUD-updates ziet — enkel de commando-bevestigingen.

7.1. Evaluatie

- **Demonstratie van het project en bespreking van de code gebeurt op het examen.**
- **De cijfers voor de demonstratie van het project tellen mee voor permanente evaluatie.**
- **Cijfers voor de bespreking van de code en architectuur tellen mee voor het examen.**